Constraint-Based Problem Solving

University Exam Timetabling as a Constraint Satisfaction Problem (CSP)

PROBLEM SET-UP

Five courses, Artificial Intelligence (A), Database Systems (B), Computer Networks (C), Operating Systems (D) and Software Engineering (E), must be scheduled into 3 exam slots (Slot1, Slot2, Slot3). Courses with shared students cannot share a slot. The conflicting pairs are A–B, A–C, B–C, B–D, C–D, C–E and D–E.

- Search uses the static variable order **A → B → C → D → E** and the value order **Slot1 → Slot2 → Slot3**, as required.
- An **assignment attempt** is one try of a value for a variable. A **rejected attempt** violates a constraint. A **backtrack** is the undoing of an earlier accepted assignment.

(A): PEAS DESCRIPTION

PEAS	Description for the exam-scheduling agent
Performance measure	Every course gets exactly one slot. No two conflicting courses share a slot (zero student clashes). Only the 3 available slots are used. Low computation effort (few assignment attempts and backtracks).
Environment	5 courses, 3 slots (one exam per day over three days), 7 conflict pairs from student enrolment overlaps. Fully observable, deterministic, static, discrete, single-agent, sequential (a CSP).
Actuators	Assigns or changes a slot for a course, and outputs or publishes the timetable (notice, database update, notification to students and staff).
Sensors	Reads the course list, the slot list, the enrolment data or conflict list, and the current partial timetable (which course holds which slot).

(B): AGENT TYPE

Best choice: a goal-based agent.

Agent type	Suitable?	Reasoning

Simple reflex	No	Reacts only to the current percept. Choosing a slot depends on the other courses already scheduled, so a fixed condition-action rule cannot guarantee a conflict-free timetable.
Model-based reflex	Partly	Remembers the partial timetable but has no goal to plan towards.
Goal-based	**Yes**	The goal is explicit: a complete assignment that satisfies all 7 constraints. The agent searches (backtracking, forward checking) for a sequence of assignments that reaches this goal state.
Utility-based	Not needed	Any conflict-free timetable is equally acceptable. There are no preferences (e.g. minimise gaps) to trade off, so a utility function adds nothing.

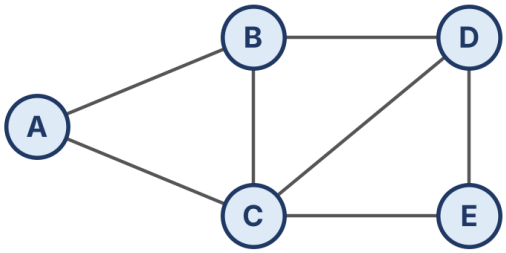
(C): CSP FORMULATION AND CONSTRAINT GRAPH

Component	Definition
Variables	X = {A, B, C, D, E}
Domains	D(A) = D(B) = D(C) = D(D) = D(E) = {Slot1, Slot2, Slot3}
Constraints	Binary inequality constraints: $A \neq B,\ A \neq C,\ B \neq C,\ B \neq D,\ C \neq D,\ C \neq E,\ D \neq E$ (7 constraints; each means "different slots".)

Constraint graph: vertices V = {A, B, C, D, E}; edges E = {A–B, A–C, B–C, B–D, C–D, C–E, D–E} (7 edges).

Vertex	Neighbours	Degree
A	B, C	2
B	A, C, D	3
C	A, B, D, E	4
D	B, C, E	3
E	C, D	2

The graph contains three triangles (A-B-C, B-C-D, C-D-E), so at least 3 slots are needed. With 3 slots a solution can exist, and C is the most constrained variable (degree 4). The graph is drawn below:



Edges in the drawing: A–B, A–C, B–C, B–D, C–D, C–E, D–E.

(D): PLAIN BACKTRACKING SEARCH

Variables are assigned in the order A, B, C, D, E and values are tried as Slot1, Slot2, Slot3. A value is checked against the already-assigned neighbours only.

Step	Attempt	Result	Reason
1	A = Slot1	Accept	No neighbour assigned yet
2	B = Slot1	**Reject**	Conflict with A = Slot1 (A–B)
3	B = Slot2	Accept	A = Slot1 ≠ Slot2
4	C = Slot1	**Reject**	Conflict with A = Slot1 (A–C)
5	C = Slot2	**Reject**	Conflict with B = Slot2 (B–C)
6	C = Slot3	Accept	A = Slot1, B = Slot2, both different
7	D = Slot1	Accept	B = Slot2 and C = Slot3, both different (D is not linked to A)
8	E = Slot1	**Reject**	Conflict with D = Slot1 (D–E)
9	E = Slot2	Accept	C = Slot3 and D = Slot1, both different. All 5 courses assigned.

Final timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2.

Slot	Courses
Slot1	Artificial Intelligence (A), Operating Systems (D)
Slot2	Database Systems (B), Software Engineering (E)
Slot3	Computer Networks (C)

Counts: 9 assignment attempts (5 accepted, 4 rejected) and **0 backtracks**. With this ordering every dead end was resolved by trying the next value of the same variable, so no earlier assignment had to be undone. The result is verified: all 7 pairs have different slots.

(E): BACKTRACKING WITH FORWARD CHECKING

After each assignment, the assigned value is removed from the domains of its unassigned neighbours. If any domain becomes empty, the assignment fails at once. Domains are written as sets of slot numbers (1 = Slot1, and so on). The starting domains are A,B,C,D,E: {1,2,3} each.

Step	Assignment	Pruning done	Domains after (A, B, C, D, E)
1	A = Slot1	Remove 1 from B and C	A{1} B{2,3} C{2,3} D{1,2,3} E{1,2,3}
2	B = Slot2	Remove 2 from C and D	A{1} B{2} C{3} D{1,3} E{1,2,3}
3	C = Slot3	Remove 3 from D and E	A{1} B{2} C{3} D{1} E{1,2}
4	D = Slot1	Remove 1 from E	A{1} B{2} C{3} D{1} E{2}
5	E = Slot2	No unassigned neighbours left	All variables assigned

Notes: in step 1, B and C are pruned because they neighbour A, but D and E are untouched. In step 2, B = Slot2 reduces C to a single value {3} immediately. In step 3, the domain of D is forced to {1} and E is cut to {1,2}. In step 4, E is forced to {2}, so its value is known before it is even tried.

Final timetable: A = Slot1, B = Slot2, C = Slot3, D = Slot1, E = Slot2 (the same as plain backtracking).

Counts: 5 assignment attempts, 0 rejected attempts and **0 backtracks**.

(F): COMPARISON

Criterion	Plain Backtracking	Backtracking + Forward Checking
Assignment attempts	9	5 (one per variable)
Rejected (failed) attempts	4 (B=Slot1, C=Slot1, C=Slot2, E=Slot1)	0
Backtracks (undone assignments)	0	0
Final timetable	A1, B2, C3, D1, E2	A1, B2, C3, D1, E2 (identical)

Work per assignment	Checks only against assigned neighbours, when the value is tried	Prunes neighbours' domains after each assignment (extra bookkeeping)
Detects dead ends	Only after trying the bad value	Early, as soon as a domain becomes empty

Forward checking reduces the search effort:

- It removes inconsistent values from neighbours' domains in advance, so bad values such as B = Slot1, C = Slot1, C = Slot2 and E = Slot1 are never tried. This is why the failed attempts drop from 4 to 0.
- It exposes forced moves: C's domain becomes {3} and E's becomes {2}, so each of these variables has exactly one candidate left.
- It detects failure early. If a domain became empty, the algorithm would abandon that branch immediately, instead of discovering the dead end several levels deeper. On harder instances, this prevents large amounts of wasted work and many backtracks.

Overall efficiency: both methods return the same valid timetable and neither needed a backtrack on this small instance, because the static order happened to lead straight to a solution. Forward checking is nevertheless more efficient: 5 attempts against 9 (about 44% fewer) and no rejected tries. The price is a small amount of pruning work per assignment, which is worthwhile because it reduces the number of search nodes. On larger or tighter CSPs, the advantage grows, and it can be improved further with MRV/degree ordering or arc consistency (AC-3).

APPENDIX: PYTHON CODE AND OUTPUT

PYTHON CODE:

CO1 AT2 CODE.txt - C:/Users/RUCHITA/Documents/AI LAB PROGRAMS/CO1 AT2 CODE.txt (3.14.7)

File Edit Format Run Options Window Help

```
"""
Exam timetabling as a CSP: plain Backtracking vs Backtracking + Forward Checking.
Variables: A..E (courses). Domain: Slot1..Slot3. Constraint: conflicting pairs differ.
Static variable order A,B,C,D,E ; value order Slot1,Slot2,Slot3.
"""
VARS = ["A", "B", "C", "D", "E"]
SLOTS = ["Slot1", "Slot2", "Slot3"]
EDGES = [("A", "B"), ("A", "C"), ("B", "C"), ("B", "D"),
         ("C", "D"), ("C", "E"), ("D", "E")]
NEIGH = {v: set() for v in VARS}
for u, v in EDGES:
    NEIGH[u].add(v)
    NEIGH[v].add(u)


def backtracking():
    stats = {"attempts": 0, "failed": 0, "backtracks": 0}
    step = [0]
    print(f"{'Step':<5}{'Attempt':<14}{'Result':<44}")

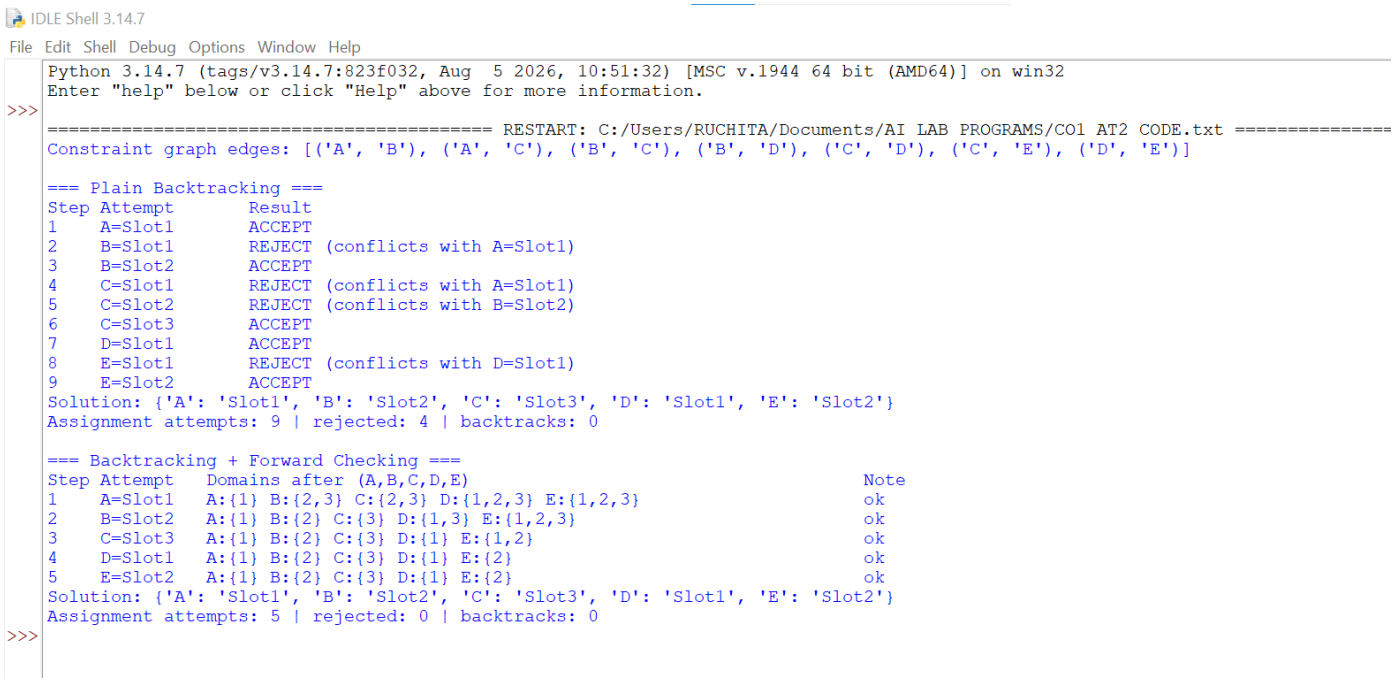
    def log(txt, res):
        step[0] += 1
        print(f"{step[0]:<5}{txt:<14}{res}")

    def solve(assign, i):
        if i == len(VARS):
            return assign
        var = VARS[i]
        for val in SLOTS:
            stats["attempts"] += 1
            clash = [n for n in NEIGH[var] if assign.get(n) == val]
            if clash:
                stats["failed"] += 1
                log(f"{var}={val}", f"REJECT (conflicts with {clash[0]}={val})")
                continue
            assign[var] = val
            log(f"{var}={val}", "ACCEPT")
            result = solve(assign, i + 1)
            if result:
                return result
            del assign[var]
            stats["backtracks"] += 1
        return None

    init = {v: list(SLOTS) for v in VARS}
    return solve({}, init, 0), stats


if __name__ == "__main__":
    print("Constraint graph edges:", EDGES)
    print("\n=== Plain Backtracking ===")
    sol, st = backtracking()
    print("Solution:", sol)
    print("Assignment attempts:", st["attempts"], "| rejected:", st["failed"],
          "| backtracks:", st["backtracks"])
    print("\n=== Backtracking + Forward Checking ===")
    sol2, st2 = forward_checking()
    print("Solution:", sol2)
    print("Assignment attempts:", st2["attempts"], "| rejected:", st2["failed"],
          "| backtracks:", st2["backtracks"])
```

PROGRAM OUTPUT:



CONCLUSION:

The exam timetabling task was modelled as a CSP with five variables, three-slot domains and seven inequality constraints, and solved by a goal-based scheduling agent. Both plain backtracking and backtracking with forward checking produced the same conflict-free timetable: **Slot1: A and D; Slot2: B and E; Slot3: C**. Forward checking needed only 5 assignment attempts compared with 9, and avoided all 4 rejected attempts by pruning domains in advance. It is therefore the better method for this problem and the more scalable choice for larger timetabling problems.